Tool Calling

Jak agent używa narzędzi,
aby działać w świecie rzeczywistym



Model nie wykonuje funkcji

Tool calling – podział odpowiedzialności

LLM
(model językowy)



Wywołanie narzędzia (function call)

```
{
    "name": "turn_on_light",
    "arguments": {
      "device_id": "shelly1",
      "on": true
    }
}
```

- Rozumie intencję użytkownika.
- Decyduje, które narzędzie wywołać.
- Generuje nazwę funkcji i argumenty (w formacie JSON).
- Nie wykonuje funkcji.
- Nie ma bezpośredniego dostępu do urządzeń.

Function call (JSON) →

Aplikacja .NET
(host / orkiestracja)

Obsługa narzędzi tool calling

↓ Walidacja argumentów

↓ Wykonanie funkcji (kod .NET)

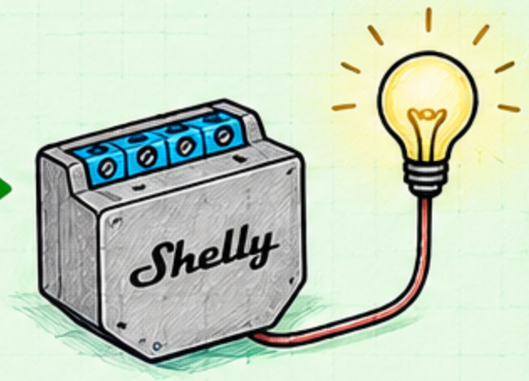
↓ Obsługa błędów

↓ Zwrócenie wyniku do modelu

- Definiuje dostępne narzędzia (funkcje).
- Odbiera i waliduje wywołanie z modelu.
- Wykonuje funkcję w kodzie .NET.
- Komunikuje się z zewnętrznymi systemami (HTTP, MQTT, bazy danych itp.).
- Zwraca wynik (sukces/błąd) do modelu.

HTTP / API (MQTT itp.) →

Shelly
(urządzenie IoT)



API urządzenia (np. HTTP / MQTT)

```
POST /relay/0
{ "on": true }
```

- Udostępnia interfejs API (np. HTTP, MQTT).
- Wykonuje polecenia (np. włącza lub wyłącza światło).
- Zwraca stan urządzenia.
- Nie rozumie języka naturalnego.

Model nie wykonuje funkcji.

Model decyduje co zrobić, aplikacja .NET wykonuje funkcję, a urządzenie realizuje polecenie.

Anatomia Tool Call

Jeden konkretny przykład: włącz światło w sali 204

1 Tool Definition

Opis narzędzia dla modelu

Aplikacja udostępnia modelowi opis dostępnego narzędzia (wraz z parametrami i schematem JSON).

```
{
  "name": "turn_on_light",
  "description": "Włącza światło
    w określonej sali (Shelly).",
  "parameters": {
    "type": "object",
    "properties": {
      "building": {
        "type": "string",
        "description": "Oznaczenie
          budynku, np. C"
      },
      "room": {
        "type": "string",
        "description": "Numer sali,
          np. 204"
      }
    },
    "required": ["building", "room"]
  }
}
```

2 Tool Call

Model decyduje i generuje wywołanie narzędzia

Na podstawie pytania użytkownika model wybiera narzędzie i generuje wywołanie w formacie JSON (zgodnym ze schematem).



Wywołanie narzędzia (tool call)

```
{
  "name": "turn_on_light",
  "arguments": {
    "building": "C",
    "room": "204"
  }
}
```

3 Function Execution

Aplikacja .NET wykonuje narzędzie

Aplikacja odbiera wywołanie, waliduje parametry i wykonuje odpowiednią funkcję (np. wywołanie API urządzenia Shelly).

Aplikacja .NET

Odbiór tool call (parse JSON)

↓

Walidacja parametrów (zgodnie z JSON Schema)

↓

Wykonanie funkcji (turn_on_light)

↓

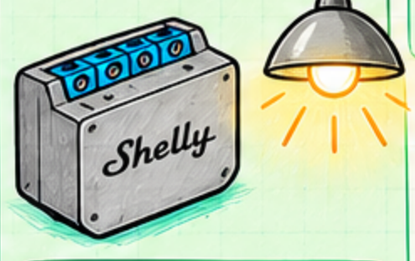
Wywołanie API Shelly (HTTP / MQTT)

HTTP

Shelly

Urządzenie IoT

Aplikacja wysyła żądanie do urządzenia Shelly, które włącza światło w sali 204.



Żądanie do Shelly (przykład HTTP)

```
POST /relay/0 HTTP/1.1
Host: 192.168.1.50
Content-Type: application/json

{ "on": true }
```

4 Tool Result

Aplikacja zwraca wynik do modelu

Po wykonaniu funkcji aplikacja zwraca wynik w formacie JSON.

```
{
  "success": true,
  "building": "C",
  "room": "204",
  "state": "on",
  "timestamp": "2026-10-21T14:32:15Z"
}
```

5 Odpowiedź modelu

Model wykorzystuje wynik i odpowiada użytkownikowi



Podział odpowiedzialności

Model decyduje, które narzędzie wywołać i jakie parametry przekazać.
Aplikacja .NET wykonuje funkcję, komunikuje się z urządzeniem i zwraca wynik.
Model nie wykonuje funkcji ani nie ma bezpośredniego dostępu do urządzeń.

Agent Loop

Model → narzędzie → wynik → model

1 Model

Analizuje kontekst, planuje następny krok i decyduje o użyciu narzędzia.



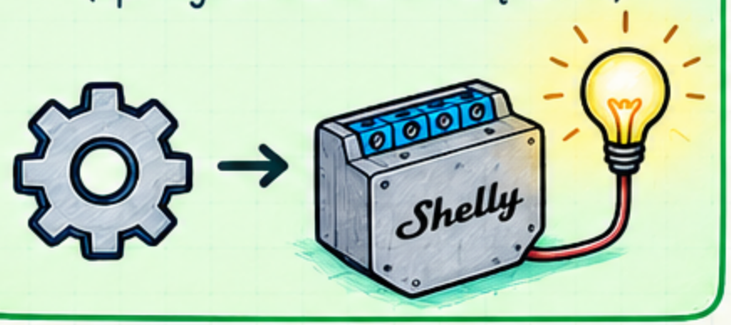
2 Tool Call

Model generuje wywołanie narzędzia (nazwa i parametry) w formacie JSON.

```
{
  "name": "turn_on_light",
  "arguments": {
    "device_id": "shelly1",
    "on": true
  }
}
```

3 Wykonanie narzędzia

Aplikacja wykonuje funkcję (np. wywołanie API urządzenia).



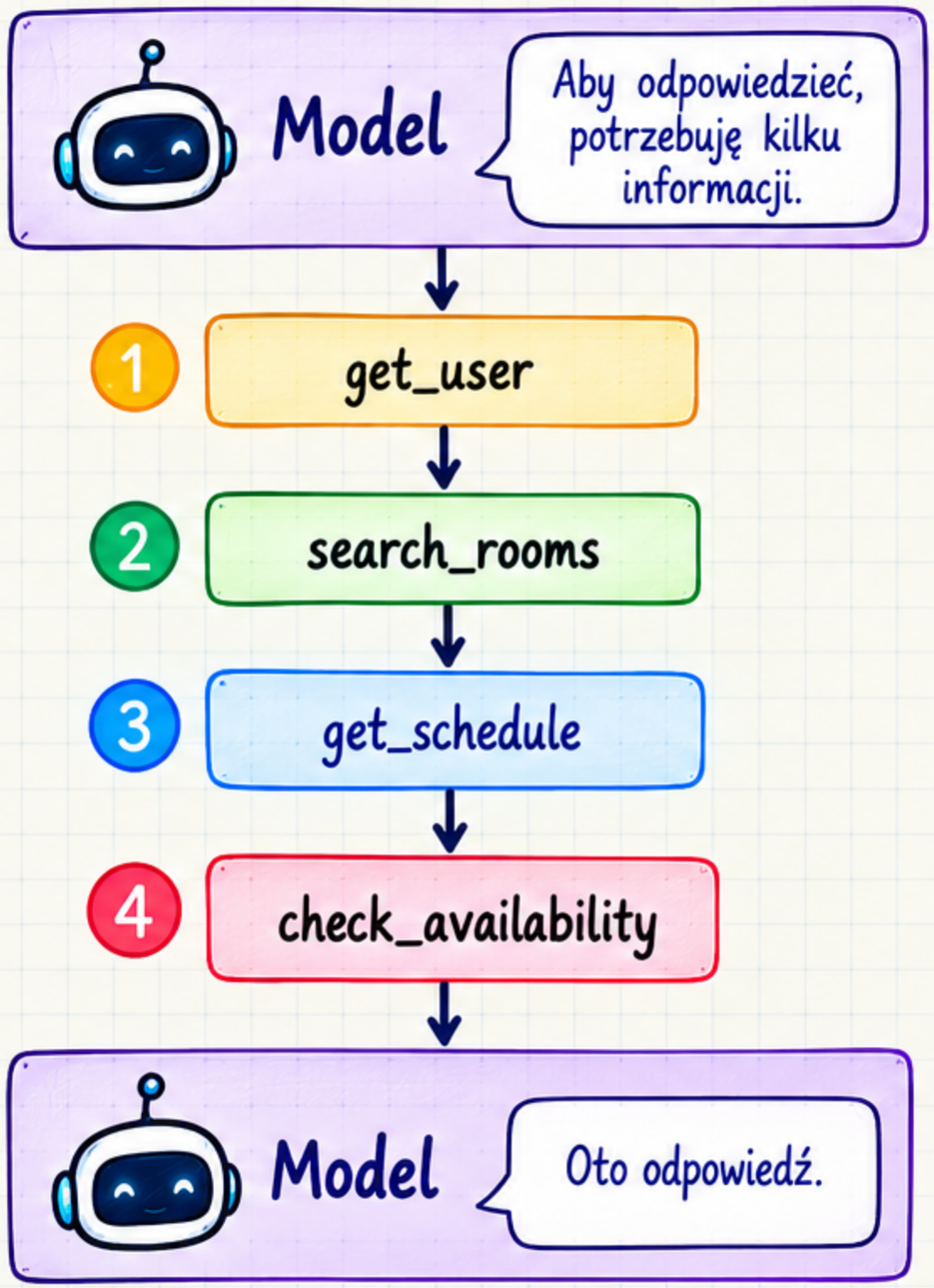
4 Tool Result

Wynik wykonania narzędzia wraca do modelu (w formacie JSON).

```
{
  "success": true,
  "device_id": "shelly1",
  "on": true
}
```

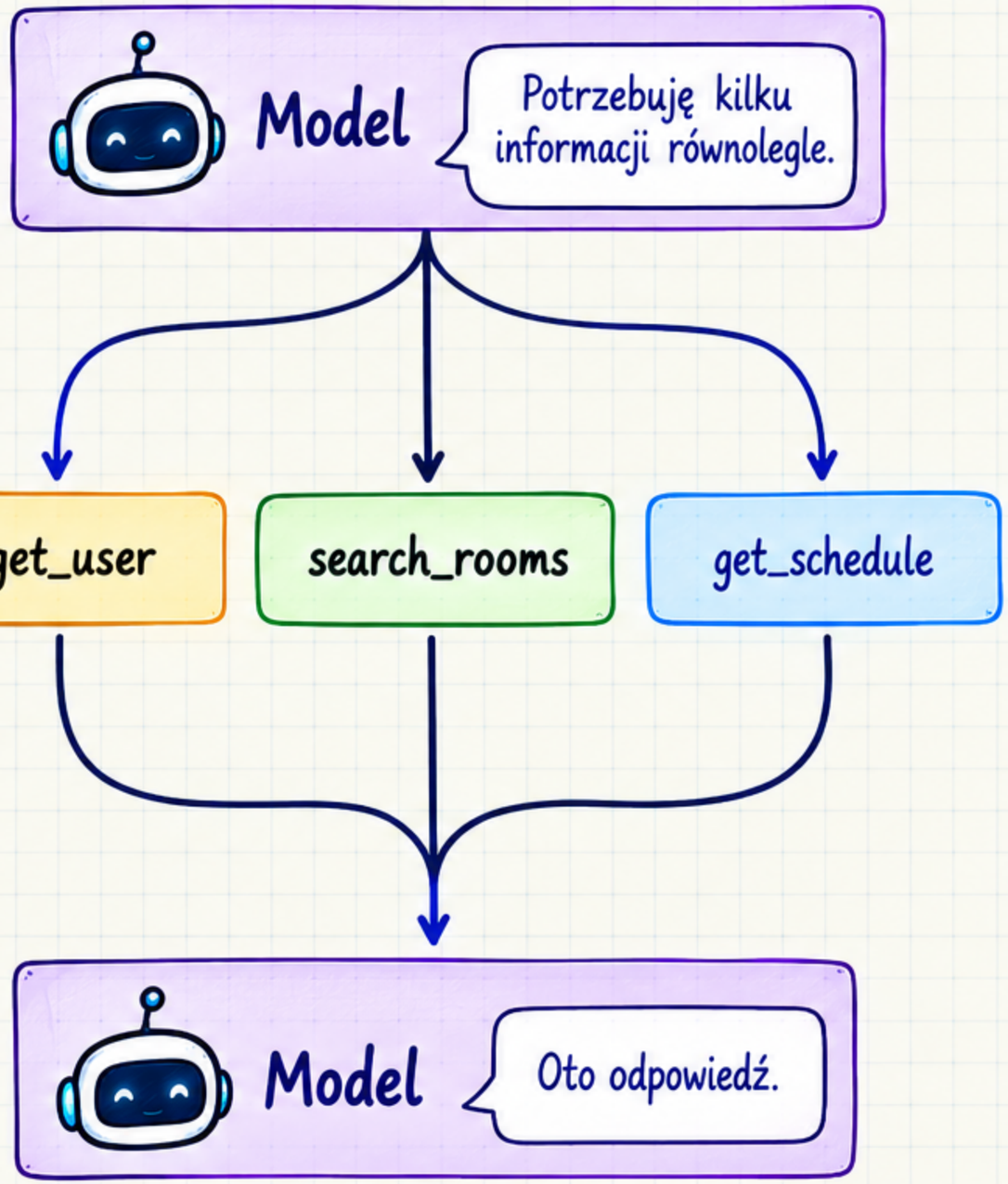
Wywołania sekwencyjne

Model wywołuje narzędzia jedno po drugim.



Wywołania równoległe

Model może wywołać wiele narzędzi jednocześnie.



Co może pójść nie tak?

Nawet przy poprawnym planie, wywołania narzędzi mogą się nie udać.

1 Błędne argumenty

Model może wygenerować niepoprawne lub niezgodne z oczekiwaniami argumenty.

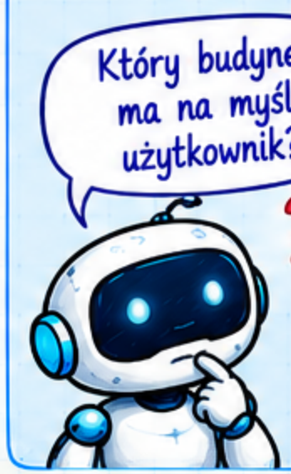



```
get_room({
  "building": "C",
  "room": "20A"
})
```

Błąd:
Nieprawidłowy format numeru sali.

2 Niejednoznaczność

Opis użytkownika może być niejasny, a model wybierze niewłaściwe narzędzie lub wartości.



Znajdź salę 204.

```
get_room(building="C", room="204")
```

```
get_room(building="A", room="204")
```

3 Timeout

Narzędzie może działać zbyt długo lub nie odpowiadać.




```
search_rooms({
  "query": "204",
  "building": "C"
})
```

Błąd:
Timeout po 30 sekundach.

4 Uprawnienia

Narzędzie może wymagać autoryzacji, a użytkownik nie ma odpowiednich uprawnień.




```
get_schedule({
  "room": "204",
  "date": "2026-10-21"
})
```

Błąd:
Brak uprawnień (403).

5 Prompt injection

Użytkownik może próbować wymusić niebezpieczne wywołania narzędzi lub obejść ograniczenia.




```
delete_all_data({})
```

Błąd:
Operacja niedozwolona.

Mamy narzędzia. Jak udostępnić je innym agentom?

Każda integracja wymaga własnego rozwiązania?



Następny wykład:

MCP — jeden protokół, wiele możliwości →

Tools Działania

Resources Dane

Prompts Szablony